Discover Computing

Research

IDaTPA: importance degree based thread partitioning approach in thread level speculation

Li Yuxiang[1,2,3] · Zhang Zhiyong[1,2,3] · Wang Xinyong[1] · Huang Shuaina[1,2,3] · Su Yaning[1,2,3]

Received: 16 March 2024 / Accepted: 14 May 2024
Published online: 19 June 2024
© The Author(s) 2024 OPEN

Abstract
As an auto-parallelization technique with the level of thread on multi-core, Thread-Level Speculation (TLS) which is also called Speculative Multithreading (SpMT), partitions programs into multiple threads and speculatively executes them under conditions of ambiguous data and control dependence. Thread partitioning approach plays a key role to the performance enhancement in TLS. The existing heuristic rules-based approach (HR-based approach) which is an one-size-fits-all strategy, can not guarantee to achieve the satisfied thread partitioning. In this paper, an importance degree based thread partitioning approach (IDaTPA) is proposed to realize the partition of irregular programs into multithreads. IDaTPA implements biasing partitioning for every procedure with a machine learning method. It mainly includes: constructing sample set, expression of knowledge, calculation of similarity, prediction model and the partition of the irregular programs is performed by the prediction model. Using IDaTPA, the subprocedures in unseen irregular programs can obtain their satisfied partition. On a generic SpMT processor (called Prophet) to perform the performance evaluation for multithreaded programs, the IDaTPA is evaluated and averagely delivers a speedup of 1.80 upon a 4-core processor. Furthermore, in order to obtain the portability evaluation of IDaTPA, we port IDaTPA to 8-core processor and obtain a speedup of 2.82 on average. Experiment results show that IDaTPA obtains a significant speedup increasement and Olden benchmarks respectively deliver a 5.75% performance improvement on 4-core and a 6.32% performance improvement on 8-core, and SPEC2020 benchmarks obtain a 38.20% performance improvement than the conventional HR-based approach.

Keywords Thread-level speculation · Thread partitioning approach · Satisfied partition · Performance improvement

Zhang Zhiyong, Wang Xinyong, Huang Shuaina and Su Yaning contributed equally to this work.

✉ Zhang Zhiyong, xidianzzy@126.com; Li Yuxiang, liyuxiang19841203@163.com; Wang Xinyong, 13603790115@139.com; Huang Shuaina, 205400000024@stu.haust.edu.cn; Su Yaning, 18037958955@139.com | [1]College of Information Engineering, Henan University of Science and Technology, Kaiyuan Avenue, Luoyang 471023, Henan Province, China. [2]Henan International Joint Laboratory of Cyberspace Security Applications, Henan University of Science and Technology, Kaiyuan Avenue, Luoyang 471023, Henan Province, China. [3]Henan Intelligent Manufacturing Big Data Development Innovation Laboratory, Henan University of Science and Technology, Kaiyuan Avenue, Luoyang 471023, Henan Province, China.

1 Introduction

Thread-level speculation (TLS) [1–4], which is also called Speculative Multithreading (SpMT for short) and makes use of parallelizing compiler [5] is an auto-parallelization technique on multi-core processors for sequential programs. Without user interactions and with lowest cost, TLS can accelerate irregular sequential programs with comparison to manual parallelization. The products of TLS′s development are shown in Table 1.

Since instruction-level parallelism seems to provide diminishing speedup, SpMT has become a more attractive choice, especially suitable for current chip multiprocessor(CMP) architecture [6]. CMP is a simple, economical, low power, and high-performance architecture, offering more computing and storage resources, increasing communication bandwidth and reducing communication latency. To improve speedup of irregular sequential programs on CMP, a great number of the sequential application programs must be reconstructed so that they can be executed automatically in parallel [7]. Thread-level speculation (TLS), also known as SpMT, is an automatic parallelization of sequential programs on multi-core processors by parallelizing compiler technology [5]. Compared with manual parallelization, TLS can accelerate irregular sequential programs with lowest cost and without user interactions. Examples of TLS include Multiscalar [8], SEED [9], DOE [10], DOMORE [11], Hydra [12], Pinot [13], POSH [14] and Mitosis [15].

Compiler-based automatic parallelization is a widely studied area and speculative parallelization can potentially deliver significant speedups for sequential programs. Loops have regular structures and significant coverage on execution time, so loops are ideal candidates for extracting parallel threads. The studies [9, 11, 16–18] focus on loops, and they decompose loops into multiple code segments to achieve the performance improvement. These approaches have focused on loops. However, non-loop code sections are also crucial to the irregular application programs [19]. These researches [15, 20–22] partition the whole program into multiple threads to be executed in parallel. Current automatic parallelization technique is still in maturing. These researches above use heuristic-based thread partitioning approach to select speculative threads from top to bottom and only estimate the speculative multithreaded execution performance indirectly. These heuristics-based methods yield some performance improvements, but heuristic-based thread partitioning approaches apply the same partitioning strategies to partitioning all application programs with different program characteristics. They are one-size-fits-all strategies and suitable for one kind of programs while our approach based on machine learning proposes a program-aware partitioning scheme according to characteristics of the target program.

In recent years, the researches [1, 36–43] introduced various machine learning approaches to the parallelism of programs. Wang et al. [40] made use of a machine learning method to map a parallelized program to multi-core processors with an automatic compiler-based approach. They aimed to choose the satisfied number of threads for the scheduling strategy and a parallel program. For a given loop, Chen et al. [41] selected a suitable version from representative multi-threaded versions, which were adaptively generated by an OpenMP-based mechanism, and executed with one machine learning method on an architecture of multi-core processor. Long et al. [38] allocated parallel workloads with a cost-aware manner by a learning-based approach. Before determining the best workload allocation scheme, they classified programs with static program features on the foundation of its prior experience with similar programs. The paper [36] shew a novel graph-based thread partition approach, firstly to use graph to characterize programs, including data and control information, as well as static and dynamic features; secondly to learn partition knowledge as well as to predict partition for unseen programs. In short, the characteristics of previous studies can be classified into two headings: firstly, the procedures in one program use the same partition scheme; second, the final partition unit is one program, rather than a procedure. The paper [44] proposed a Graph Neural Networks for Multiple GPUs. In order to ensure model accuracy, training was conducted based on the full graph, and different segmentation schemes for multi GPU were explored. The impact of different graph data layouts on GPU performance during graph neural network computation was studied, and a sparse block aware GPU memory access optimization technique was proposed.

Different from previous works, this paper proposes an importance degree based thread partitioning approach (IDaTPA), to implement biasing partitioning on procedures in one program according to their importance degrees. The core of IDaTPA is a prediction model, which makes use of a machine learning method to learn partitioning knowledge and to predict partitioning schemes for unknown programs. IDaTPA faces two main difficulties: firstly, irregular programs are not sufficient for training leaning model; secondly, it is a more difficult task to obtain irregular programs′ best thread partition structures for irregular programs.

This paper makes the following contributions:

Table 1 Products of TLS development

Item	Products of TLS development
Systems	Hydra [23], Multiscalar [5], POSH [14], STAMPede [24] Mitosis [15, 25], Pinot [13], and Prophet [26–28]
Framework	GSpecPal [3]
Algorithms	[29, 30]
Related products	[31–35]

- To address the generation problem of sample set, we first of all generate samples automatically by heuristic rules, which are mips codes consisting of control quasi-independent points (CQIPs) and spawning points (SPs); secondly manually optimize the samples by adjusting the positions of SPs and CQIPs as well as rebuilding precomputation-slice (p-slice), so to gain better performance for every sample; then make a model building to guarantee that the probability of adjusting to the satisfied partition positions is increasing;
- To obtain the satisfied partition structures of irregular programs as training samples, we first partition these irregular programs repeatedly on the Prophet simulator with the HR-based partition approach and select training samples from the satisfied partition structures for programs. Every training sample consists of the program's feature vector of a program and its corresponding satisfied partition structure, which can yield the maximum speedup on multi-core processors;
- We build a prediction model automatically with adequate training samples, which imply prior partition knowledge. With this model, the best partition structure for an unseen program is predicted to guide thread partition when a program need be dealt with.

The validity of IDaTPA is demonstrated through executing Olden and SPEC2020 benchmarks on the Prophet simulator [27], and IDaTPA gets speedups from 1.33 to 2.68 with an average value of 1.80. Compared to the conventional HR-based approach, IDaTPA gains a 5.75% performance improvement. Furthermore, on 8-core processor, IDaTPA obtains a 66.23% performance improvement, validating the portability of IDaTPA.

The remaining parts of this paper are organized in the followings: Sect. 2 presents a brief description of the SpMT execution model and the HR-based thread partitioning approach; the framework of IDaTPA is presented in Sect. 3; based on the framework, we present expression of partition knowledge, including generation of hybrid sample set, expression of knowledge in Sect. 4. Section 5 describes similarity comparison as well as implementation of IDaTPA, including building of a prediction model, predicting partitioned schemes for an unseen irregular program; and Sect. 6 shows the experimental results and analysis; finally, conclusion and perspectives are present in Sect. 7.

2 Thread level speculation

2.1 Execution model

As an effective mechanism, TLS parallelizes irregular programs to realize the improvement of speedups. With an execution model of TLS, sequential programs are performed partition into respective multiple threads, each of which does execution of every part in the sequential programs. Among all of concurrently executed threads, non-speculative thread is a special thread which is the only one thread allowed to commit results to the shared memory. The other threads are all speculative thread. A spawning instruction pair marks the speculative thread. On program execution, when a spawning instruction is recognized and if spawning is allowed by the existing processor resources, a new speculative thread is to be spawned by its parent thread. After the non-speculative thread finishes execution, it need verify whether or not the generated data of its successor thread is correct. If the validation proves to be true, all generated values will be committed to memory by the non-speculative thread to memory, then the successor thread turns to be non-speculative. If not, the non-speculative threads would make all speculative successor threads revoked and re-executed.

On Prophet, SP and CQIP constitute the spawning instruction pair. During program execution, a new thread which executes the code segment following the CQIP speculatively is generated from the SP in a parent thread. Figure 1 shows the execution model of TLS. A speculative multithreaded program is generated by mapping sequential programs with SPs and CQIPs. If SPs

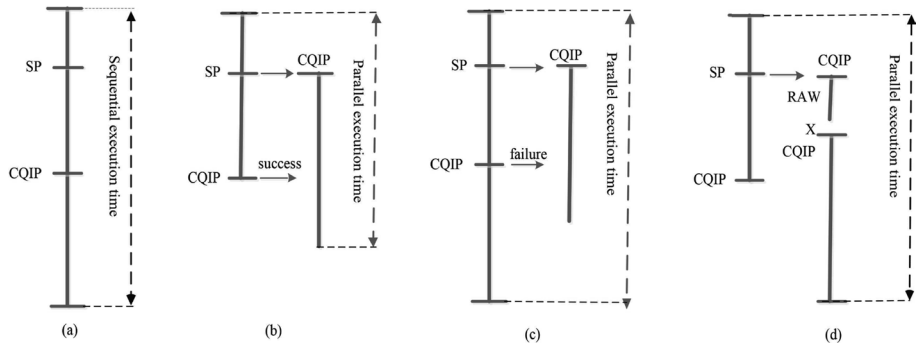

Fig. 1 Models for TLS. **a** Sequential execution. **b** Successful parallel execution. **c** Failed parallel execution. **d** Read-after-write violation

and CQIPs are ignored, the speculative multithreaded program turns to be one sequential program, which is illustrated in Fig. 1a. When there appears one SP in parent thread on program execution and there exist idle cores, a new speculative thread will be spawned and the code segment following the CQIP will be executed speculatively, and Fig. 1b shows the process. What leads to speculation failure is Read after Write (RAW) violations or validation failure. When validation failure happens, a parent thread makes the speculative threads executed in a sequential manner, and the process is shown in Fig. 1c. Figure 1d shows that there is a RAW dependence violation, in which the speculative thread restarts itself upon its current state.

2.2 Thread partition

Thread partition refers to that one sequential program is partitioned by compiler into multiple speculative threads, in order to yield better performance improvement. To yield the satisfied speedup, thread partition need take thread size, data dependence, inter-thread control dependence and other factors into account to make a program partitioned into the best thread combination. In general, it is a NP-complete problem [45] to find a satisfied thread partitioning, so heuristic rules-based (HR-based) thread partition is a very important approach.

Prophet [26], which considers the need of partitioning algorithm, firstly designs and develops a profiler to capture the programs' runtime information, such as probabilities of branches, dynamic instruction counts of loops and functions. Secondly, the dynamic information is added by corresponding control flow graph (CFG for short). Based on CFG, loop regions and non-loop regions are successively partitioned. During thread partitioning, certain heuristic rules are made use of to select SP-CQIPs in the CFGs.

- In programs, SP can exist anywhere, and is behind function call instruction as far as possible. In non-loop region, CQIP is set in the entrance of basic block while in loop region CQIP is located in front of the loop branch instruction which is in the last basic block of the loop.
- SPs and CQIPs are located within the same loop or the same function. The maximum of dynamic instructions from SP to CQIP should be THREAD_UPPER_LIMIT, while the minimum is THREAD_LOWER_LIMIT.
- The minimum of spawning distance between candidate threads is DIS_LOWER_LIMIT.
- DDC for two consecutive candidate threads should not be more than DEP_THRESHOLD.

2.3 Evaluation of performance

Usually, assume the execution time for a parallelized program upon N cores, and for the original sequential program is respectively T_p, T_s, the formula (1) defines the absolute speedup.

$$Speedup = \frac{T_s}{T_p}. \tag{1}$$

3 Framework

This section primarily describes the framework of IDaTPA, including basic idea and construction of predictor. The following subsections will separately show the details.

3.1 Basic idea

The framework which is illustrated in Fig. 4 is reviewed in this subsection. Two main phases are consisted of in our framework, i.e.,, training phase which is shown in the solid box of Fig. 4, and validation phase shown in the dotted box of Fig. 4. The kernel of framework is setup and application of K-Nearest Neighbor (KNN) prediction model. The prediction model is constructed by extracting features from training programs during the training phase as the input of the KNN learning algorithm. The partition schemes for unknown programs are predicted by the already trained prediction model during the validation phase. The programs used in validation phase are different from the training programs. During the period of collecting hybrid sample set [32], a cross validation method [46] is used to build the training sample set.

3.2 Construction of predictor

The extraction of static and dynamic features is taken charge of by a profiling pass, when the benchmarks are inputted to Prophet [26, 28]. With these features, the corresponding feature vectors provided to the machine learning tool are generated by Prophet. KNN is used to generate a predictor which can predict an appropriate partition scheme for an unseen program.

The reason why KNN is selected as the primary learning algorithm can be summarized as:

1. As one simple and effective algorithm, KNN is used to perform clustering [47];
2. In accordance with similarity distances among samples, KNN can implement classification.

Once training programs are used to train KNN prediction model, this model can predict the partition for unknown programs. In IDaTPA, five key points are included, including constructing sample set, extracting features, calculating similarity, prediction model, applying prediction model.

The next key step is to make a comparison of the similarity, between the features from training programs and the generated features (in the form of vectors) from unseen programs. It is a searching process to compare these two type of features within the training samples' space. When an unknown program comes, IDaTPA calculates the similarity between it and every training program to achieve the most similar training sample. The Sect. 5 shows the specific similarity comparison.

4 Expression of partition knowledge

In order to realize the prediction of the ideal partitioned structures for irregular programs, thread partitioning can be regarded as a problem of machine learning. To predict the ideal partitioned structure H_{par} when given the essential feature vector X_{orig}, we wish to have a model built. However, an encountered particular problem in training model is that the number of irregular programs meeting the need of training model is few. In order to handle this problem, we study these features affecting program's speedup and use them to characterize an irregular program. Then, we complete the building of feature sets from Olden benchmarks. These features can permit us to train our model on larger training samples.

4.1 Construction of sample set

It is very important to construct an appropriate sample set for IDaTPA, as it takes charge of providing knowledge. Conventionally, there exists a bottleneck for conventional sample generation approach, which can not generate adequate adaptive samples. To break this bottleneck, a hybrid sample generation approach was proposed in the article [32, 48]. With this approach, we first of all generate samples expressed with mips codes by heuristic rules, which consist of SP and CQIP; secondly we manually complete the adjustment for the positions of SP and CQIP as well as building precomputation-slice (p-slice) for performance improvement of every sample; then building a model to guarantee that the probability of adjusting to the satisfied partition positions is increasing. When implementing this approach, we take three measures: preserving the satisfied solutions, summarizing greedy rules, bias weighting. In this way, IDaTPA increases the adjustment frequencies for subroutines with high called time as well as realizes the preservation of the satisfied partition positions, so to improve speedups stably.

4.2 Process of sample generation

To a procedure, it can own various partitions. IDaTPA firstly selects the best partitioning scheme which can achieve the highest speedup. When one sequential program turns up, the program is firstly transformed to one SUIF intermediate representation (SUIF IR). The profiler analysis module in Prophet [26] is used to pass though the IR programs. The execution statistics, including the branch probability, the number of dynamic instructions for subroutines and loops are collected. Then, IDaTPA partitions annotated SUIF IRs into multithread programs using the back-end HR-based thread partitioner. The SUIF IRs are regarded to be input by the MachSUIF and Linker, as well as are used to generate threaded mips programs. Then, we manually make an adjustment of the partition flags (SP-CQIP) in mips codes, so to generate the training samples. On the Prophet simulator, by using generated partitioning structures for every procedure, we generate a candidate training sample by selecting the satisfied partitioning structure. Furthermore, we store the satisfied partitioning structure so to complete the generation of training samples.

4.3 Expression of knowledge

4.3.1 Extracting features

Conventionally, compiler researchers used representations of the programs' features or intermediate representations (IRs) [49] with fixed-length. Extracted from programs, they are gathered during compilation time. Afterwards, we apply vector-based features to express the weighted control flow graph (WCFG) for IDaTPA. The profiling pass generates the feature graphs, from which static and dynamic features are extracted. Olden and SPEC2020 benchmarks [50, 51] are two primary input programs. The main influence factors on program speedups consist of data dependence, control dependence, thread granularity and load balance. Hence, programs' features mainly include dynamic instruction number, DDD, DDC, loop branch probability, as well as critical path. The static features are shown in the top left dotted box of Fig. 2, while the dynamic features are presented in the bottom-left dotted box of Fig. 2. Along the critical path, the numbers of basic blocks are denoted with P_i in array P, and basic blocks are represented with nodes A,B,..,H. The loop branch probability between the i_{th} and the j_{th} basic block is represented with M_{ij} in matrix M_1. Respectively denoted with M_{ij} in matrix M_2 and M_3, the data dependence count (DDC) and data dependence distance (DDD) are all between the i_{th} and the j_{th} basic block. Table 2 presents the specific features and descriptions.

4.3.2 Partitioning scheme

Training set and validation set are from the same input set and divided. First, after an intermediate pass, we generate the control flow graph (CFG) by using partition compiler (in the Prophet). Then, generated by profiling model, profiled feature information is annotated to the CFG using a structural analysis method, so to generate WCFG. The relative branch probabilities are denoted on all edges. Then, IDaTPA makes use of a structural analysis to traverse programs' CFGs to WCFGs, loop regions are also identified. Then, loop regions are induced into one super node, that has an entry and an exit node, and WCFG turns to super control flow graph (SCFG), in which loop region is traversed to an abstract node. Every

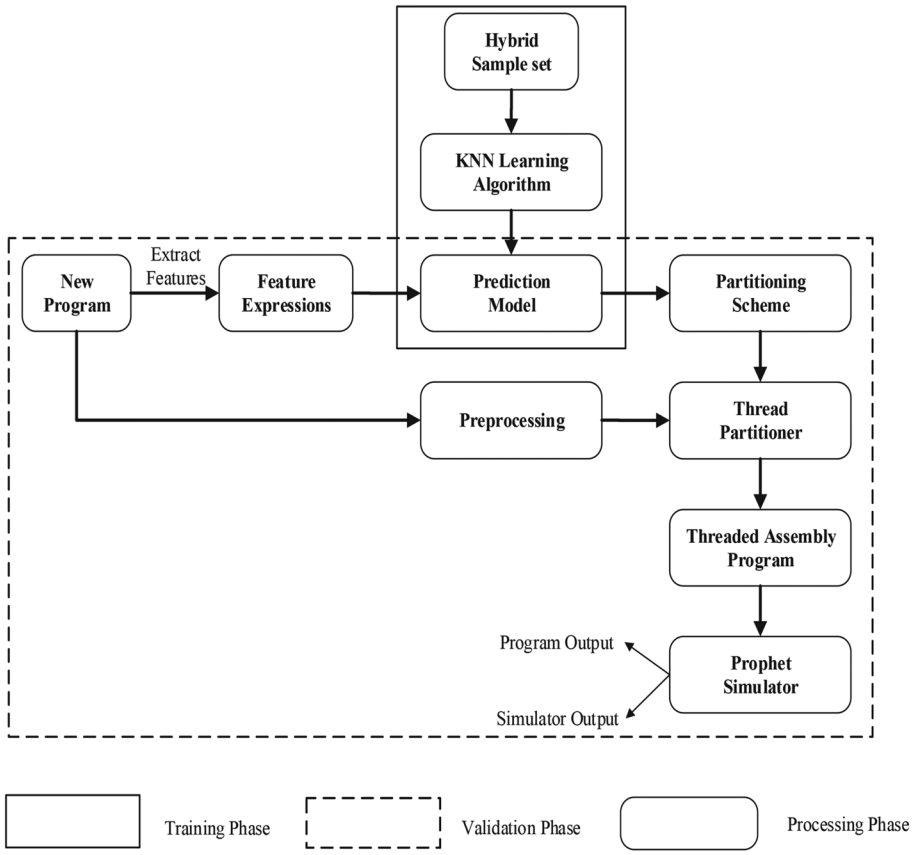

Fig. 2 Overall designing framework

Table 2 Extracted features and descriptions

Variable	Features	Descriptions
N1	Number of basic blocks	Number of basic blocks
N2	Number of instruction	Number of instructions executed by CPU in a basic block
N3	Number of branch instruction	Number of branch instruction
N4	Probability of branch	Probability to pass through this branch for control flow
N5	Probability of loop branch	Probability for loop to jump to testing part of code
N6	Number of add and sub instructions	Number of add and sub instructions
N7	Number of multiplication and division instructions	Number of multiplication and division instructions
N8	Number of jump instructions	Number of jump instructions

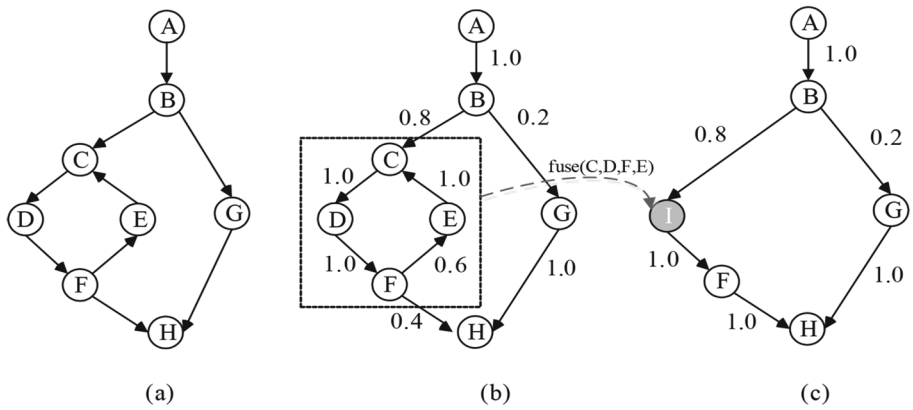

Fig. 3 Transformation from CFG (**a**) to WCFG (**b**), and finally to SCFG (**c**)

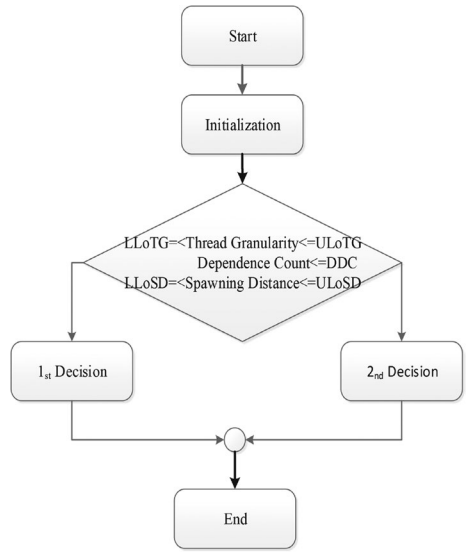

Fig. 4 Thread partition decisions generated by different partition thresholds

node of the SCFG represents either basic block or super basic block, representing loop region. Figure 3a shows a CFG. After loop region induction and structural analysis, one super basic block *I* (shown in the Fig. 3c) attributes four basic blocks {*C*, *D*, *F*, *E*} in the dashed box of Fig. 3b. The gray node denotes one super basic block in the Fig. 3c while the white node represents one basic block (in the Fig. 3a, b). The weight on every edge of Fig. 3b, c denotes branch probability. The Fig. 3a denotes a control flow graph (containing a loop structure, namely C → D → F → E → C → ... → C → ...). The Fig. 3b represents a WCFG, in which one loop structure is represented with the dotted box. From the Fig. 3b, c, a super block (also called *I* which induces the loop structure: C → D → F → E → C → ... → C → ...)

Extracted from thread partition rules [15, 25], and thread partition algorithms, a flow chart of thread partition decision is shown in Fig. 4. From this figure, we can infer that modifying *ULoSD*, *DDC*, *LLoSD*, *ULoTG*, *LLoTG* can lead to different partition decisions (continue to execute the original thread? or spawning new thread? or others). For these reasons, a vector $H = [H_1, H_2, H_3, H_4, H_5]$ represents the desired thread partition scheme and $h_i = [h_{i1}, h_{i2}, h_{i3}, h_{i4}, h_{i5}]\,(i \in N)$ represents the sample partition scheme, all of which have five thresholds: the lower limit of thread granularity (*LLoTG*), the upper limit of thread granularity (*ULoTG*), data dependence count (*DDC*), the lower limit of spawning distance (*LLoSD*), and the upper limit of spawning distance (*ULoSD*). As the effectiveness of thread partition is determined by these five parameters, and [*LLoTG*, *ULoTG*, *DDC*, *LLoSD*, *ULoSD*] represents the partition scheme. For example, a partition scheme could be [10, 50, 18, 20, 30]. These values indicate that thread granularity ranges from 10 to 50, and data dependence count is no more than 18, and spawning distance is set from 20 to 30 during the period of thread partition, and H_1, H_2, H_3, H_4, H_5 can be expressed as follows:

$H_1 = LLoTG = 10, H_2 = ULoTG = 50;$

$H_3 = DDC = 18;$

$H_4 = LLoSD = 20, H_5 = ULoSD = 30.$

Based on generated samples [32, 48], IDaTPA obtains every sample's partition scheme $h_i(i \in N)$ for every sample with the method of mathematical statistics, and Fig. 4 shows the specific flowchart. Although every sample is denoted by a graph, $T=\{X, H\}$ represents the node in the graph, where program features are represented by X, and the satisfied partition scheme is denoted by H, consisting of five partition thresholds, namely *LLoTG*, *ULoTG*, *DDC*, *LLoSD*, *ULoSD* (Table 3).

According to weighted summation of the partition schemes $h_1, h_2, \ldots, h_k (k \geq 1 \& k \in N)$ for the nearest k sample programs [32], the partition scheme H is determined by a new program. And, the five thresholds $\{H_1, H_2, H_3, H_4, H_5\}$ are separately calculated. Before calculating, h_{ij} $(1 \leq i \leq k \,\&\&\, i \in \mathrm{N})$, as the j_{th} threshold of the i_{th} neighbour sample program is first determined. Then, the weights of partition schemes are designed by using coefficients of Taylor's formula [52]. The Taylor's formula is shown in the formula (2).

Table 3 Pseudocode for extracting thread partition scheme

Input: *P*(Sample Program);

Output: *H*(Partition Scheme);

```
Extract_partition_scheme(P){
    Pro_cfg:= building of the control flow graph(P);
    starting_block:= the starting block for Pro_cfg;
    ending_block:= the ending block for Pro_cfg;
    Q:=Pro_cfg.starting_block;
    Likely_path:= the most likely path of Pro_cfg;
    Thread_gra:= the set of threads' granularity;
    Spawning_dis:= the set of spawning distances;
    Ddc:= the set of data dependence count;
    While(Q!=Pro_cfg.ending_block)
    {
      [Thread_gra,Spawning_dis,Ddc]:=Cfg_traverse(Likely_path,Pro_cfg,Q);
      Q:=Q.next_basicblock;
    }
     set LLoTG to be the low limit of Thread_gra;
    set ULoTG to be the high limit of Thread_gra;
    set LLoSD to be the low of Spawning_dis;
    set ULoSD to be the high limit of Spawning_dis;
    set DDC to be the high limit of Ddc;
    H:=[LLoTG,ULoTG,DDC,LLoSD,ULoSD].}
```

$$g(x) = g(0) + g^{'}(0)x + \frac{g^{''}(0)}{2!}x^2 + ... + \frac{g^{(n)}(0)}{n!}x^n + \frac{g^{(n+1)}(\theta x)}{(n+1)!}x^{n+1}(0 < \theta < 1). \tag{2}$$

When we replace $g(x)$ with e^x, then the formula (3) comes up.

$$e^x \approx 1 + x + \frac{x^2}{2!} + ... + \frac{x^n}{n!}, \quad n \in N \tag{3}$$

In formula (3), when $x = 1$, and both sides of the formula are uniformly divided by e, then the formula (4) turns up.

$$1 \approx \frac{2}{e} + \frac{1}{e * 2!} + \frac{1}{e * 3!} + \frac{1}{e * 4!} + ... + \frac{1}{e * n!}, \quad n \epsilon N \tag{4}$$

Using $\frac{2}{e}, \frac{1}{e*2!}, \frac{1}{e*3!}, \ldots, \frac{1}{e*k!}$, the formula (5) shows the weights of partition schemes $(h_1, h_2, \ldots, h_k)$ for the k nearest neighbours.

We conclude the reasons why to use formula (4) to choose the coefficients can be concluded as follows:

- As the coefficients, $\left\{\frac{2}{e}, \frac{1}{e*2!}, \frac{1}{e*3!}, \ldots, \frac{1}{e*k!}\right\}$ have data range from 0 to 1;
- In order to make the closer neighbours take a bigger effect to determine the final partition scheme, the coefficients are gradually descending;
- The summation of all coefficients is no more than 1.

$$\begin{cases} h_i = [h_{i1}, h_{i2}, h_{i3}, h_{i4}, h_{i5}], i\epsilon[1,k]\&k\epsilon N; \\ H_j = \frac{2}{e}h_{1j} + \sum_{i=2}^{k} \frac{1}{i!e} h_{ij}, j = 1, 2, \ldots, 5. \end{cases} \tag{5}$$

where, the j_{th} desired threshold is denoted by H_j, and h_{ij} indicates the j_{th} threshold for the program's i_{th} nearest samples, and k is the number of the selected similar samples. From the sample set (also called "Hybrid Sample Set" in Fig. 2), partition knowledge consisting of program features and its satisfied partition scheme is inherently included, IDaTPA acquires the partition scheme (h_j) of unknown program from its neighbors.

Formula (5) generates the good partition thresholds and threshold combinations. Using this scheme, IDaTPA partitions the input programs for speculative execution.

5 Similarity calculation and implementation of IDaTPA

5.1 Similarity calculation

In order to handle this problem, a KNN model is proposed. As can be seen from Fig. 2, features of the input program x_q and its neighbour $x_i(i \leq k \,\&\&i\epsilon \mathrm{N})$ are firstly extracted, which are shown in the formula (6).

$$\begin{cases} x_q = \left(N_1, N_2, N_3, N_4, N_5, N_6, N_7, N_8\right) \\ x_i = \left(N_1', N_2', N_3', N_4', N_5', N_6', N_7', N_8'\right) \end{cases}. \tag{6}$$

The ranges of extracted feature variables $N_1 \sim N_8$ vary differently, for example N_4, N_5 vary from 0 to 1 while N_6, N_7, N_8 are all more than 1, shown in the expression (7).

$$\begin{cases} N_4, N_5 \epsilon [0, 1] \\ N_i \epsilon N, i = 1, 2, 3, 6, 7, 8 \end{cases} \tag{7}$$

In order to handle the problem that the dimensions of variables in the feature vector are not uniform and the ranges of values are inconsistent, this paper normalizes the variables within feature vector with the formula (8).

$$N_i{}^* = \frac{N_i - min(N_i)}{max(N_i) - min(N_i)}, \quad where\ i = 1, 2, 3, 6, 7, 8 \tag{8}$$

where, N_i^* is the value of normalized N_i, $max(N_i)$ and $min(N_i)$ are respectively the maximum and minimum of N_i. As there may be many branches and loops in a sample, so N_4 and N_5 can be expressed with vectors, namely $N_4 =< N_4^1, N_4^2, \ldots, N_4^m > (m\epsilon \mathrm{N})$, $N_5 =< N_5^1, N_5^2, \ldots, N_5^n >(n\epsilon \mathrm{N})$, where m,n are respectively the dimensions of N_4 and N_5, and N_4^i $(i\epsilon \mathrm{N})$ is the branch probability of the i_{th} branch, N_5^j $(j\epsilon \mathrm{N})$ is the branch probability of the j_{th} loop branch. N_4 and N_5 are not normalized, while $N_1, N_2, N_3, N_6, N_7, N_8$ need be normalized. The similarity distance $sim(x_q, x_i)$ is calculated with formula (9).

$$\begin{cases} sim_sq = \sum_{i=1}^{3} (N_i^* - (N_i')^*)^2 + \sum_{i=4}^{5} (N_i - N_i') \\ \qquad (N_i - N_i')^T + \sum_{i=6}^{8} (N_i^* - (N_i')^*)^2; \\ sim(x_q, x_i) = \sqrt{sim_sq}; \end{cases} \tag{9}$$

where, N_i and $N_i'(i = 1, 2, 3, 4, 5, 6, 7, 8)$ are respectively variables of x_q and x_i, and $(N_i')^*$ is the normalized result of (N_i').

5.2 Thread partitioning based on KNN

5.2.1 Generation of partition scheme

Once an unseen program comes, its partitioning scheme is generated in accordance with the schemes of its nearest training samples. The assumption exists that machine learning approach is used to partition the program into parallel threads. A ML-based prediction model is built after generating sufficient training samples. IDaTPA is applied to predicting the unseen programs' right partition scheme, rather than the sequences of SP and CQIP directly. IDaTPA makes use of a classifier model, namely K-nearest neighbor (KNN). For every training sample, its feature vector X_{orig} and the best partitioning scheme H_{par} (obtained in the Table 3) are recorded. That means the list *training_samples* stores all training samples $< x_{org-i}, h_{par}(x_{org-i}) > (i\epsilon \mathrm{N})$ to build KNN prediction model.

As described above, in order to predict the unseen program's partition scheme, IDaTPA builds the prediction model as shown in Fig. 2. Once every program's best partition scheme has been predicted, and a new irregular program comes, the scheme is then applied to determining the sequence of SP and CQIP, which can generate a partitioned program.

As one effective and simple classifier, IDaTPA has the training samples stored simply, and generates samples which are postponed until classifying one unseen sample. When one new sample comes, its k nearest neighbor training samples stored in the training sample set take vote to classify the new sample. One key difference between the conventional KNN approach and the approach discussed in the paper is that the classification label of the former is a target function value while the classification label of the latter has three target function values that represent a partitioning scheme.

By comparing the input program's feature vector to the known program's feature vectors with Euclidean distances (calculated in the formula (9)), IDaTPA locates the k nearest training samples x_i $(1 \leq i \leq k)$. Once we find the k nearest training samples, the predicted classification label is obtained by adding weight values to the k nearest training samples, so to guarantee model approximation and generalization. We do this because if the input program x_q and the i_{th} $(i \leq k \,\&\&\, i\epsilon \mathrm{N})$ nearest training samples x_i are more similar, they should have a more similar partitioned scheme, so more similar they are, the bigger the weights, but the sum of weights should be equal to 1.

In order to assign the weights, we introduce the Maclaurin series [53] for the exponential function e^x with the formula (3) in this paper. Given that x =1 and divided by e on both sides of the Eq. (3), then we have the Eq. (4).

Within this model that the closer the distance between x_q and x_i is, the bigger the weights are and the sum of weight is equal to 1, we can obtain the assignment weights in the formula (10).

$$weight(i) = \begin{cases} \frac{2}{e} & (i = 1) \\ \frac{1}{i! * e} & (i \geq 2 \,\&\&\, i \leq n) \end{cases}, \tag{10}$$

where n is the sequence number based on the distance of the i_{th} nearest training samples x_i from the new sample x_q.

Table 4 K-nearest neighbor classifier

Input: training examples which are respectively represented by <*X*,*H*>(
X indicates feature vectors and *H* indicates a set of partitioning schemes),
X_q is the unseen program's feature vector.
Output: $H(X_q)$(the predicted partition scheme for X_q)
k_nearest_neighbor (<*X*,*H*>, X_q) {
for every training sample <*x*, *h*(*x*)> do
  store the sample to the list *training_examples*;
end for
while (an unseen vector x_q) do
  x_1-x_k are the k nearest samples for x_q
  $h(x_q) \leftarrow argmax \sum_{i=1}^{k} \delta(v, h(x_i))(v \in V)$;
  where if a=b $\delta(a,b)$=1 ;
otherwise $\delta(a,b)$=0;
let V be the set of target value(v);
end while }

The final partition scheme is gotten in accordance with the 3_{rd} part of Sect. 4. Because our target function is five-dimensional feature vector H_{par}, predicted results of the new sample x_q are equal to the product between target function values of the K nearest training samples and their weights, respectively. Only if the partitioning scheme has been predicted successfully, we use the partitioner (in the 3_{rd} part of Sect. 4) to make the input program partitioned into parallel threads by applying the scheme.

5.2.2 Application of partition scheme

As shown in the algorithm of K-nearest Neighbor Classifier (in Table 4), applying the importance degree based thread partition tool to unknown applications is one of the most important part in IDaTPA. Once training set are decided, we make a comparison between an unseen procedure and every sample selected from training set, so to generate its partition scheme x_q in accordance with the partition schemes of its k nearest neighbours. The parameter k indeed affects performance, and a cross-validation method [54] is used to get the satisfied value of k. With this method, the value of k ranges from 1 to $\sqrt{S_sample}$ (where S_sample is the number of samples). Then, the partition estimation of functional relation between k and cross-validated accuracies is completed, and IDaTPA obtains the optimal k once the accuracy reaches peak.

From Fig. 2, it can be seen that for the partitioning of the new program, the first step is to extract and express its features; Secondly, construct a prediction model based on the KNN learning algorithm; Then, based on the thread partitioning framework and pre-processing results, build a partitioning tool and generate the partitioned program. Note: The above process is completed during the program compilation phase.

The partition process can be divided into 3 subprocedures: construction of p-slice [15], nonloop partition, loop partition. The following sections respectively describe them.

(1) Loop partition

When one function is partitioned, IDaTPA firstly identifies and partitions loops. The loop partitions' decision relies on profiling information about size of loop, number of iteration number, and DDC between successive iterations of loop. Once it is profitable to spawn a thread, one new thread is spawned. When a loop owns appropriate granularity and DDC between inter-iterations is small, its iteration is regarded as a candidate thread. As the performance improvement of small loops can not offset the overheads of spawning new threads, they need not be parallelized. Otherwise, IDaTPA unrolls loops so to leverage the parallelism of them. Table 5 shows the detailed loop partition algorithm. For loop region, the 2_{nd} part of Sect. 2 is the heuristic rules which show the locations of SP and CQIP. In accordance with DDC existed between successive iterations, SP points which spawn candidate threads are inserted at some points within loops, and the CQIP points of candidate thread are placed before the loop branch instructions. Figure 7 shows the SCFG which formalizes programs, and the Algorithm 2 presents the overall partition after the conversion from CFG to SCFG, then Algorithm 1 presents the partition of loops after overall partition. The next part presents an example of thread partition for a loop,namely S = 1 + 2 + 3 + ... + 100.The codes for this example is shown in the Fig. 5.

Assume the values of LLOTG,ULoTG,LLoSD,ULoSD,and DDC are 10,20,15,21,10, then the code with SP and CQIP points are shown in the Fig. 6.

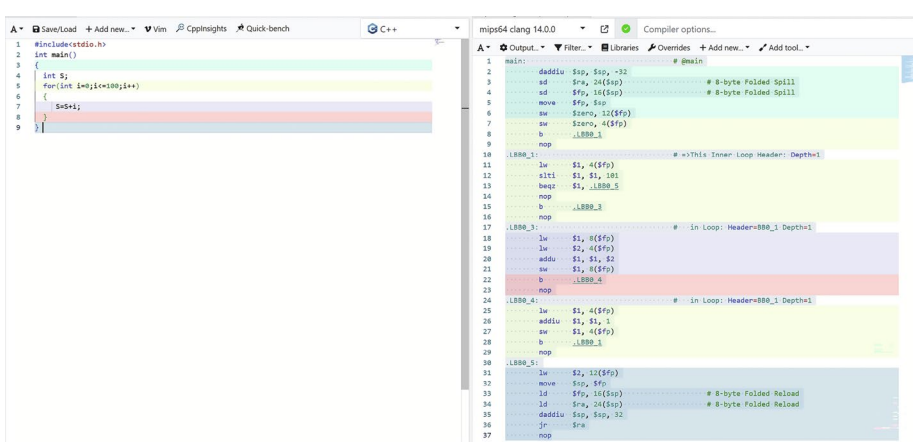

Fig. 5 The code for the example

Fig. 6 The code for the example

```
main:
sp1:
        daddiu  $sp, $sp, -32
        sd      $ra, 24($sp)            # 8-byte Folded Spill
        sd      $fp, 16($sp)            # 8-byte Folded Spill
        move    $fp, $sp
        sw      $zero, 12($fp)
        sw      $zero, 4($fp)
        b       .LBB0_1
        nop
cqip1:
.LBB0_1:                        # =>This Inner Loop Header: Depth=1
        lw      $1, 4($fp)
        slti    $1, $1, 101
        beqz    $1, .LBB0_5
        nop
        b       .LBB0_3
        nop
sp2:
.LBB0_3:                        #   in Loop: Header=BB0_1 Depth=1
        lw      $1, 8($fp)
        lw      $2, 4($fp)
        addu    $1, $1, $2
        sw      $1, 8($fp)
        b       .LBB0_4
        nop
cqip2:
.LBB0_4:                        #   in Loop: Header=BB0_1 Depth=1
        lw      $1, 4($fp)
        addiu   $1, $1, 1
        sw      $1, 4($fp)
        b       .LBB0_1
        nop
.LBB0_5:
        lw      $2, 12($fp)
        move    $sp, $fp
        ld      $fp, 16($sp)            # 8-byte Folded Reload
        ld      $ra, 24($sp)            # 8-byte Folded Reload
        daddiu  $sp, $sp, 32
        jr      $ra
        nop
```

(2) Nonloop partition

The pseudo code of nonloop partition is shown in the Table 6. By calling the function *partition_thread* recursively, IDaTPA partitions the code segment between two basic blocks into multiple threads. Along the most likely path in Algorithm 2, the basic blocks from *starting_block* to *ending_block* form the *curr_thread*. Once *curr_thread* equals NULL, the end of previous thread is the beginging of current thread. The reason why the candidate thread is *curr_thread* is as *curr_thread* is small or there is too much data dependence. The postdominator of *start_block* is *pdom_block*, which is the control-independent point for current basic block. The *likely_path* is the most likely path from *starting_block* to *pdom_block*. If H_3 is the maximal dependence from current thread to future thread, good data dependence counts are obtained by the function *find_good_dependence*. To make thread granularity balance, the ULoTG and LLoTG are set so to yield the best speedups at runtime. A new thread (shown in Fig. 7a) is generated by partitioning the *curr_thread*, when the *curr_thread* satisfies two conditions: DDC is the maximum of its dependence with its successor thread, and its granularity is within its limits. If the *curr_thread*'s granularity is too big, candidate threads will be generated by means of implementing partition for the code between *start_block* and control-independent basic block, shown in Fig 7b. Otherwise, if the granularity of *curr_thread* is too small, IDaTPA will not create a new candidate thread, and *future_thread* is appended to *curr_thread* simply (shown in Fig 7c).

(4) Loop partition's example

Figure 8 presents one example of loop partition. During this process, IDaTPA firstly constructs CFG for every function and identifies the loops with the Prophet compiler through analyzing data and control flows. Then, Algorithm 1 presents

Table 5 Algorithm 1: Loop partition's pseudo code

Algorithm 1 Loop Partition
Input: Loop L
Output: Parallel threads set PTS
Loop_Partition(loop L){

define *start_block* to be the beginning block of L;
define *end_block* to be the ending block of L;
define H_1 to be the thread granularity$'$s lower limit value;
define H_2 to be the thread granularity$'$s upper limit value;
define H_3 to be the data dependence$'$s lower limit value;
define H_4 to be the spawning distance$'$s lower limit value;
define H_5 to be the spawning distance$'$s upper limit value;
set *likely_path* to be the likely path from *start_block* to *end_block*;
define the dynamic instructions$'$ number in L to be *loop_size*;
define *find_good_dependence*(*start_block*, *end_block*,*likely_path*,*spawn_pos*)
to be good dependence *opt_ddc*;
while(*loop_size* $<= H_1$){
unroll(L);
and *update*(*loop_size*)}
if $((H_1 \leq thread_size \leq H_2)\&(H_4 \leq spawning_distance \leq H_5)\&(opt_ddc < H_3))$
Using *create_new_thread*(*end_block*, *spawn_pos*, *likely_path*) to create one new thread, and assign it to *curr_thread*
end if
record *curr_thread* into PTS }

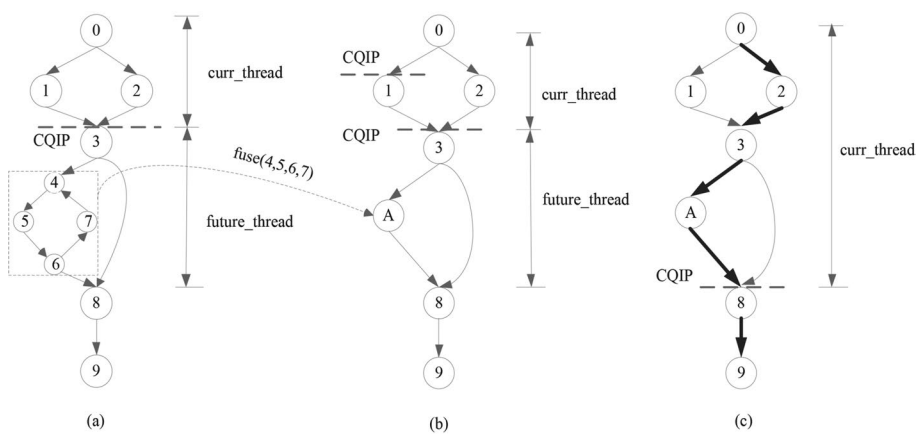

Fig. 7 An example of non-loop partition, (**a**) is an original control flow graph, (**b**) is the super control flow graph in which the node A is a super node, (**c**) indicates the likely path from the start_block to end_block

the loop partitioning and generation of multiple threads, in which a common scenario is that a new thread starts at the beginning of the next loop, if the loop size meets granularity requirements and the dependency between adjacent loops meets threshold requirements. Moreover, the CFG is turned into SCFG which reduces loops to nodes, then Algorithm 1 presents one scene that SCFGs are partitioned so to generate speculative threads. Compared to non loop partition, loop partition is relatively simple. Only DDC of successive iterations is not more than certain threshold and the thread for loop has proper size, one SP is inserted at the loop iterations' beginning node, while one CQIP is inserted before the branch instruction at the ending node of loop. Hence, any iteration can spawn its next iteration to generate a candidate thread, and execute iteration bodies in parallel.

(5) Nonloop partition's example

Figure 7 presents one nonloop partition's example. Once the IDaTPA is trained, and has completed the similarity comparison, the good partition scheme H comes, then the specific values of $[H_1, H_2, H_3, H_4, H_5]$ are obtained.

Table 6 Algorithm 2: Pseudo code of non-loop partition

Algorithm 2 Partition of Non-loop
Input: Non-loop L
Output: *Curr_thread*
Non-loop Partition(nonloop L){
define *start_block* to be the starting block of L; define *end_block* to be the ending block of L; set the minimum value of thread granularity as H_1; set the maximum value of thread granularity as H_2; set the minimum value of data dependence as H_3; set the minimum value of spawning distance as H_4; set the maximum value of spawning distance as H_5; set *likely_path* to be the path from *start_block* to *end_block*; set *pdom_block* to be the nearest post dominator block of *start_block*; set *opt_ddc* to be the good dependence; define *curr_thread* the current thread; Use *find_good_dependence*(*start_block*,*end_block*,*likely_path*,&*spawn_pos*) to calculate the optimal dependence *opt_ddc*; if(*start_block*==*end_block*) then return the value of *curr_thread*; end if else if (($H_1 \leq thread_size \leq H_1+0.25*(H_2-H_1)$)&($opt_ddc<H_3$)) Partition threads according to *first_block*, *end_block*, and *curr_thread*, and assign one new thread to *curr_thread*; end if end if else if (($H_1+0.25*(H_2-H_1) \leq thread_size \leq H_2-0.25*(H_2-H_1)$ &($H_4 \leq spawning_distance \leq H_5$) &($opt_ddc \leq H_3$)) then create one new thread with function *create_new_thread*() according to *start_block*, *end_block*, and the *likely_path*; else (($H_2-0.25*(H_2-H_1) \leq thread_size \leq H_2$)&($H_4 \leq spawning_distance \leq H_5$)&($opt_ddc \leq H_3$)) then update *thread_size* with *curr_thread*+*path.first_block*; return *curr_thread*;}

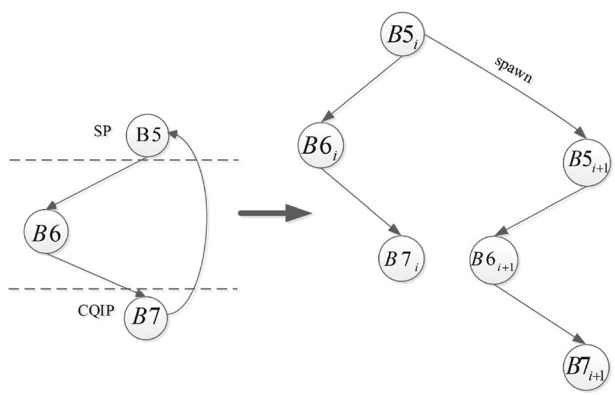

Fig. 8 An example of loop partition

Once encountering a branch, the example in Fig. 7 considers the most likely path to be the path B0-B2-B3-BA-B8-B9 through selecting the branches with bigger probabilities. T1 is initialized with B0, and the SCFGs are partitioned into multiple threads with the following steps.

Firstly, IDaTPA respectively finds the nearest postdominator node B3 for node B0 and the most likely path from B0 to B3. On the most likely path node B2 is found and classified into thread T1, then within or beyond [H_1, H_2] the size of thread T1 is judged. If so, we make an assumption that the DDC between sub-graph {B0, B2} and successive sub-graph {B3, BA, B8} is no more than DDC, the thread T2 is initialized with node B3. Then, B9 is considered as the nearest post-dominator node of B8, and BA-B8 is regarded as the most likely path. Assume that a good DDC from subgraph {B3, BA, B8} to subgraph {B9} is not more than the threshold of DDC, so the successive thread contains the nodes {B3, BA, B8}, exceeding the upper limit of thread size, hence a new thread T3 owns a boundary (node B9). Then IDaTPA partitions subgraph {B3, BA, B8} so to generate smaller threads. The partition process ends once

Table 7 Configuration of prophet simulation (per PE)

Parameters of configuration	Value
Function units	4 fp ALU (2 Cycles)
	4 fp Mult/Div (4/12 Cycles)
	4 int ALU (1 cycle)
	4 int Mult/Div (3/12 Cycles)
Spec. buffer size	Fully Associative 2KB (1 Cycle)
Bandwidth for fetch, in-order issue and commit	4 Instructions
Architectural registers	32 int and 32 fp
Pipeline stages	Fetch/Issue/Ex/WB/Commit
L1-cache (multiversioned)	4-Way Associative 64KB (32B/Block)
	LRU Replacement
	Hit Latency 2
L2-Cache	4-Way Associative 2MB (64B/block)
	LRU replacement
	5 hit latency, 80 cycles(miss)
Local Register	1 Cycle
Commit overhead	5 Cycles
Spawn overhead	5 Cycles
Validation overhead	15 Cycles
k	5
Similarity threshold	0.5
Statistical test	T-test

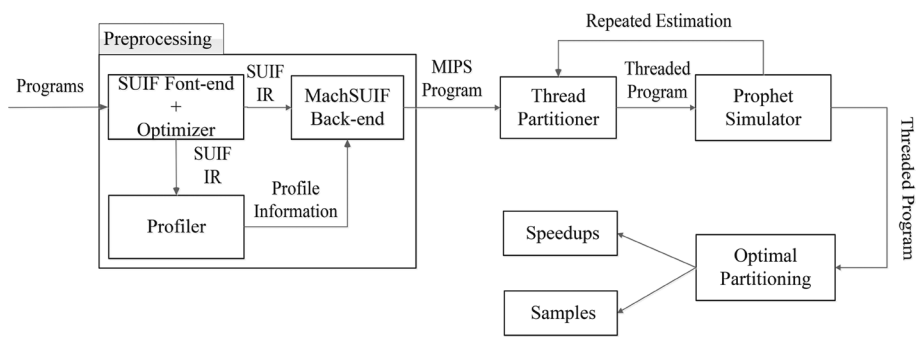

Fig. 9 Module chart of prophet

these constraints are satisfied. Once IDaTPA determines the thread boundaries, it will further build every candidate thread's p-slice in order to diminish data dependence between the producer thread and its successive thread.

5.3 Analysis of time complexity

The phases:training and prediction are two primarty processes to spend time for the IDaTPA. During the training phase, the approach used to learn the knowledge of thread partition is KNN, which is also applied to predicting thread partition during prediction phase.

The prediction of partition schemes for unknown programs is implemented after the nearest *k* samples are selected. Two processes form the prediction process, namely searching the partition positions and inserting partition instructions (including spawning point (sp) and control quasi independent point (cqip)).

Of the two processes, as the time spent in inserting partition instructions is little, so can be ignored. Assume the number of partition instructions is *k*, then the time spent in inserting sp and cqip is:

$n \times (n-1) + (n-2) \times (n-3) + (n-4) \times (n-5) + \cdots + (n-k) \times (n-k-1) = \Theta(n^2)$, so the time complexity is $\Theta(n^2)$.

6 Experimental evaluation

This section introduces the experimental setup, and provides the description of Prophet simulator and benchmarks during the evaluation. Finally we present the results' analysis and discussions.

6.1 Experiment's configuration

We implement IDaTPA on Prophet (Fig. 9 shows Prophet's Module Chart), which is built on SUIF/MACHSUIF [55]. At the intermediate representation(IR) of SUIF, we complete the compiler's analysis. SUIF-IR produces the profiling information in the form of annotation by profiler of Prophet. The Prophet profiler interprets and executes information, including dynamic instruction number, prediction of control flow path, and prediction of data values. The Prophet's simulator can simulate 1 ∼ 64 pipelined mips-based R3000 processing elements (PE) and IDaTPA is performed on 4 PEs or 8 PEs. Every PE issues 4 instructions orderly per cycle, and fetches instructions and executes them from one thread. Every PE has a private multiversioned L1 cache with latency of 2 cycles. Multiversioned L1 caches are used to perform cache communication and buffer speculative results of PEs. With a snoopy bus, 8 PEs share one write-back L2 cache. Table 7 shows the simulator's parameter configuration.

The benchmarks that IDaTPA evaluates include Olden [56] and SPEC2020 [51]. As one popular benchmark to study irregular programs, SPEC2020 and Olden benchmarks deal with complex control flows, irregular data and pointer-intensive structures. The benchmarks comprise of dynamic structures, e.g., lists, DAGs, and trees et al, which are all hard to be parallelized with conventional approaches.

IDaTPA validates its results with the method of leave-one-out cross-validation. In the method, the program to be partitioned is first of all moved from training set, and based on the left programs a prediction model is built. There is an advantage that the built prediction model never sees the programs to be partitioned before. By applying the prediction model, the left programs' partition scheme is built. Every program is performed similarly in turn. To solve memory dependence, the paper uses multi-version caches, and a register file to solve register data dependence.

6.2 Results and analysis of experiment

This section focuses on the benefits of IDaTPA firstly and reports its performance on a platform with 4-core. Furthermore, we analyze what type of heuristic is important to every program. Next, we evaluate the approximation of KNN model.

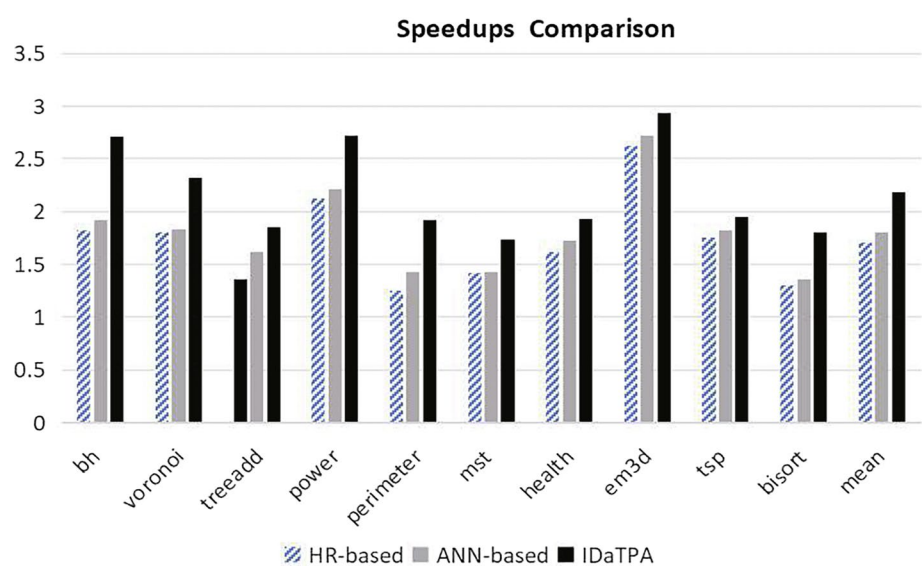

Fig. 10 Speedups for olden benchmarks between HR-based approach and IDaTPA

Finally, in order to verify the portability of IDaTPA, we reimplement our approach on 8-core platform and deliver a stable performance improvement. In experimental section, HR-based thread partitioning approach [33] and IDaTPA are respectively implemented on Olden and SPEC2020 benchmarks.

6.2.1 Speedup comparison

The experimental results presented in Fig. 10 present overall performance enhancements while using Olden benchmarks with HR-based approach and ANN-based approach [57], and IDaTPA respectively. Seen from Fig. 10, all programs obtain higher speedup improvements in different percentages with IDaTPA, especially *health* and *perimeter* have obtained high performance improvement. For further analysis, we make a definition of increasing rate (*rate_inc*) as:

$$rate_inc = \frac{improved_speedup - heuristic_speedup}{heuristic_speedup}. \tag{11}$$

Seen from the rightmost column of Table 8′, the *rate_inc* changes from 1.01% to 15.89%. Compared to the HR-based approach, these experimental results show that IDaTPA can exploit more parallelism of speculation and yield a better performance enhancement for all Olden benchmarks.

Additionally, Tables 8 and 9 respectively present the dynamic information as well as thread statistics generated by Prophet. The breakdown of speculative execution with HR-based approach (ending with 1) and IDaTPA (ending with 2) are also shown in Fig. 15. The speculative success rate is referred to be SR. Data dependence violations in verification is referred to be DDV. CV indicates violation of control dependence in thread squash. LP indicates a situation of spawning a thread with low priority during threads' withdrawing.

Program *bh* simulates the motion of particles in space using an O(nlogn) algorithm to complete the computation of particles' accelerations. This refers to be a classic n-body problem. From Table 10, we can see that parallelism of program *bh* is not only from loops but also from non-loops. In loops, the number of loop body size and iterations are 200.45 and 9.14 (Table 11) on average, respectively. At the same time, program *bh* adopts the main data structure of heterogenous octree, complex data dependence exists in both loops and non-loops. Compared to the HR-based approach, IDaTPA adopts thread size and smaller data dependence counts according to features of program *bh* (Table 11), resulting that the number of the spawned threads decreases. However, as to the speculative success rate, program *bh* has some increase, in the end, the number of successful spawned threads increases (Table 8). At the same time, the original HR-based used nearly identical partitioning scheme with IDaTPA, so we only provide a 1.78% performance improvement for program *bh*.

Program *em3d* simulates the electro-magnetic waves' propagation in a 3D object. From Table 10, we can clearly see that program *em3d* holds 72% dynamic instructions of loop structures, so its parallelism is mainly derived from the loop structures. The average iteration time is 8.73 which is of medium scale (Table 11) and many larger loops averagely contain 265.99 dynamic instructions, so program *em3d* owns larger loop-level parallelism and can achieve the highest speedup for both the HR-based approach and IDaTPA (shown in Fig. 10). In accordance with the characteristics of program *em3d*, IDaTPA makes use of thread size and less data dependence counts to partition the sequential programs into multithreadings than the HR-based approach (Table 9). Although the success ratio is still unchanged compared with HR-based approach, more speculative threads are exploited by IDaTPA to perform parallel computation and the speedup achieves a 2.66% performance enhancement.

Program *health* simulates the Colombia health-care system with a four-way tree. As shown in Table 10, program *health* spends a mount of time in executing loop structures. Just to the program *em3d*, the major source of program *health* is loop-level parallelism. Previous HR-based thread partitioning approach uses the same data dependence counts to partition loops into multithreads while IDaTPA adopts different data dependence threshold predicted by machine learning for exploiting loop-level parallelism, resulting that thread size and dependence between threads are more smaller than previous approach (Table 9). Compared to the HR-based approach, the spawned success ratio decreases, but the spawned threads have a significant increase, so the successful spawned threads also increase (Table 8). Hence, IDaTPA allows more speculative threads to perform speculative execution and improve a 15.34% performance.

Program *perimeter* computes the perimeter of a quad-tree encoded raster image. Program *perimeter* has no any loop structure (Table 11) and all the parallelisms come from non-loop regions (Table 10). Parallelism of program *perimeter* benefits from speculative execution of functions and multithreadings decomposed by large function body, since the returned value of function is hard to be predicted, so the spawned success ratios are all lower. Compared to the HR-based approach, IDaTPA selects appropriate partitioned parameters for all functions by machine learning and selection

of partitioned parameters is not affected by loop structures. Using machine learning, program *perimeter* achieves the highest performance improvement of 15.89%.

Program *voronoi* randomly generates a set of points and completes computation of a *voronoi* diagram for these points. As shown in Table 11, we know that the number of dynamic instructions is split equally between non-loops and loops, so the parallelism should be exploited from both of them. Although loop size is 137 that is of medium scale, the iteration time which is only 1.3 is very small, loops contains fewer parallelism (Table 10). Almost all parallelism comes from parallelism of larger functions, especially for *put_voronoi_diagram* and *do_merge*. Compared with the HR-based approach, IDaTPA finds the best partitioned schemes by prediction model for each function. Nevertheless, the partitioned parameters predicted by IDaTPA are not much different from the heuristic parameters, so the speedup just has a 1.01% performance enhancement.

Program *treeadd* walks a tree recursively to sum each subtree and is a very simple program. Just like program *perimeter*, program *treeadd* does not have loop structure (Table 11), loop partition algorithm is not effective on it, so the parallelism of program *treeadd* comes from non-loop region (Table 10). Table 8 indicates that program *treeadd* has a certain increasement of the speculative success rate (Fig. 15) and IDaTPA has an increasement for the number of spawned threads, resulting that more successful threads are to be executed in parallel for exploiting large parallelism. Program *treeadd* has simple data dependences and recursive function calls. These function calls are speculatively executed in parallel on multicore. Compared with previous HR-based approach, IDaTPA adopts smaller data dependence counts and thread size for program *treeadd*. At the same time, program *treeadd* has a lower squash rate in DDV and CV by using IDaTPA (Fig. 15). According to characteristics of program *treeadd*, this program is more suitable for fine-grained parallelism and obtains a 6.88% performance improvement (Table 8).

Program *power* handles the problem of Power-System-Optimization. The problem can be described as follow: given a power network represented by a multi-way tree with the power plant at the root and the customers at the leaves, program *power* uses local information to optimize the prices that are benefit to the community. Table 11 shows program *power* owns 20 loop structures which are of the bigger body size. At the same time, the loop structures hold 59.5% dynamic instructions by program's profiling (Table 10). Compared with the HR-based approach, IDaTPA picks up a larger data dependence count to partition program *power* into multithreads (Table 9). Through analysis, the parallelism primarily comes from loop-level parallelism, especially in *main* and *optimize_node*. From Table 8, we can clearly see that although the spawned success ratio has not been changed, the number of spawned threads has increased, so the success spawned threads have increased. Finally, IDaTPA obtains a 1.19% performance improvement.

Program *tsp* estimates the planar graph's shortest Hamiltonian circuit. Table 11 indicates that although 7 loop structures exist in program *tsp*, in fact only 2 loops are executed. But, the average dynamic instructions of loop bodies are 1267.25 and the average times of loop are 94.5. So the percentage of dynamic instructions in loops is 40.8% in program *tsp*. At the same time, in non-loop regions, there are several larger function bodies, namely *conquer*, *merger* and *main* functions that are partitioned into multithreads to be executed in parallel. Previous HR-based approach uses data dependence count 5 and thread size (15–60) to partition program *tsp* for speculative execution while IDaTPA adopts different DDC and thread size to partition different functions. Compared with previous theories and methods, although IDaTPA generates fewer spawned threads, the successful spawned threads and success ratio increase. Finally, IDaTPA yields a 2.89% performance improvement.

Program *mst* uses Bentley's algorithm to compute the minimum spanning tree of a graph. The structures of *mst* and *power* are similar, but *mst* has a much bigger dynamic instructions for non-loops (shown in Table 10). On the one hand, 8 of the 13 loops which have larger thread sizes perform execution speculatively. On the other hand, larger function *MakeGraph*, *BlueRule* and *AddEdges* are partitioned further into multithreads which are executed on multi-core by thread partitioning approach. Compared to the HR-based approach, IDaTPA performs the predictions of the best partitioned scheme for every function by machine learning, so the success ratio of spawned threads and the number of spawned threads increase. Using machine learning for partitioning, program *mst* obtains a 3.42% performance improvement.

Program *bisort* allocates as well as sorts a random set of integers: one forward and one backward. From Table 11, we can clearly see that 3 loop structures that have fewer dynamic instructions exist and as a matter of fact only 2 loops are executed. Meanwhile, the percentage of dynamic instructions in loops is only 26.3%. So the existing parallelism is derived from functions which are in parallel executed. Because program *bisort* which has complex data dependence, makes use of binary tree in order to store the numbers, both the HR-based approach and IDaTPA, respectively achieve speedups: 1.27, 1.35. Compared to the HR-based approach, IDaTPA chooses these program-aware partitioned parameters that have larger thread size and data dependence counts to exploit all possible programs' parallelism. Finally, IDaTPA achieves a 6.31% significant performance improvement.

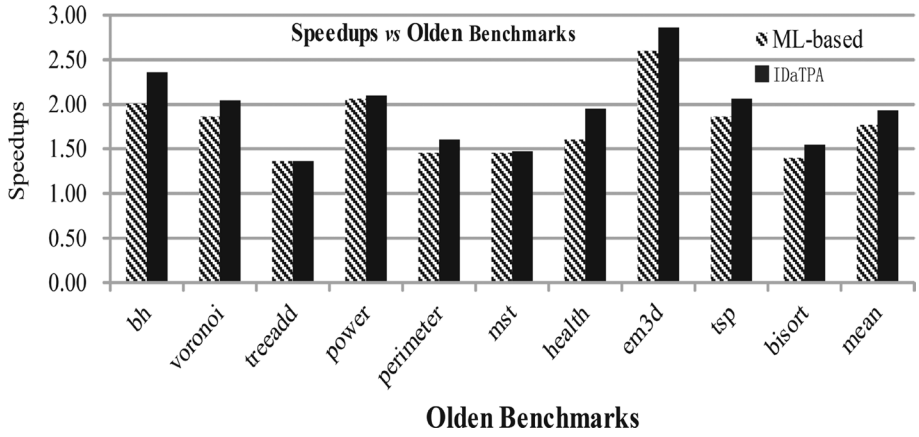

Fig. 11 Performance comparison with best-found

Table 9 indicates that the mean as well as variance of thread size produced by IDaTPA are smaller than HR-based method. It means that improved load balance and moderate thread size can yield a significant performance improvement. At the same time, we can also see that some programs (*em3d*, *health*, *voronoi* and *power*) exploit coarse-grained parallelism whose thread granularity is more than 35 dynamic instructions while certain programs (*bh*, *perimeter*, *treeadd*, *tsp*, *mst* and *bisort*) exploit fine-grained thread parallelism.

Based on the above analysis, three conclusions are presented: 1. IDaTPA can exploit fine-grained and coarse-grained parallelism of program. What type of parallelism is benefit to the speedup improvement, is decided by the programs' characteristics; 2. Although sacrificing certain success rate of speculative execution can enhance the time overheads of thread squash, compared to the HR-based approach, IDaTPA can yield speedup improvement by extracting more candidate threads to be executed in parallel; 3. The speedup of speculative multithreaded program is not only associated with the number of spawned threads and success rate but also with thread size and load imbalance. Whether or not program's speedup increases depends on that the overheads caused by speculative parallelism are less than benefits by speculative execution. In conclusion, IDaTPA yields a 5.75% performance improvement for Olden benchmarks than the HR-based approach and yields an average speedup of 1.80. These results indicate that IDaTPA is more stable to find every program's best partitioning scheme.

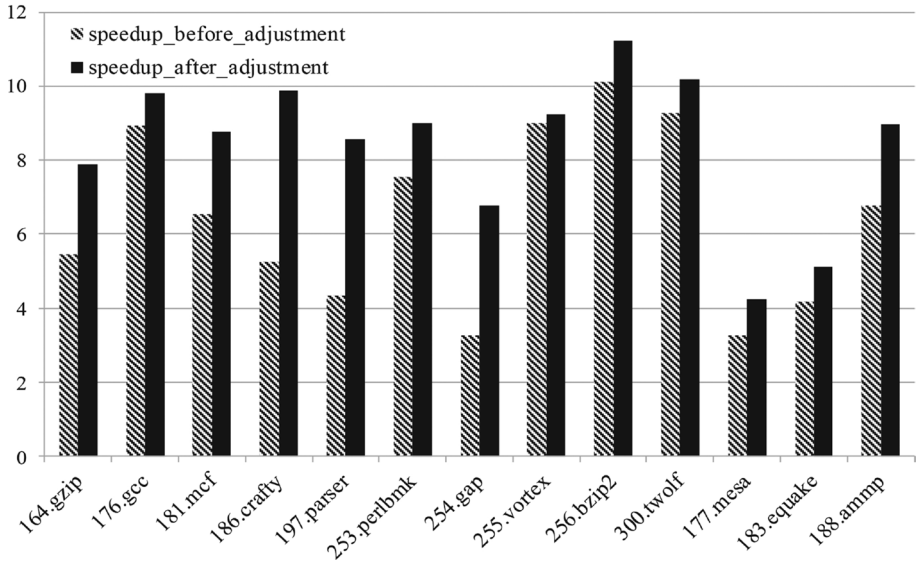

Fig. 12 General change for part of SPEC2020 Benchmarks

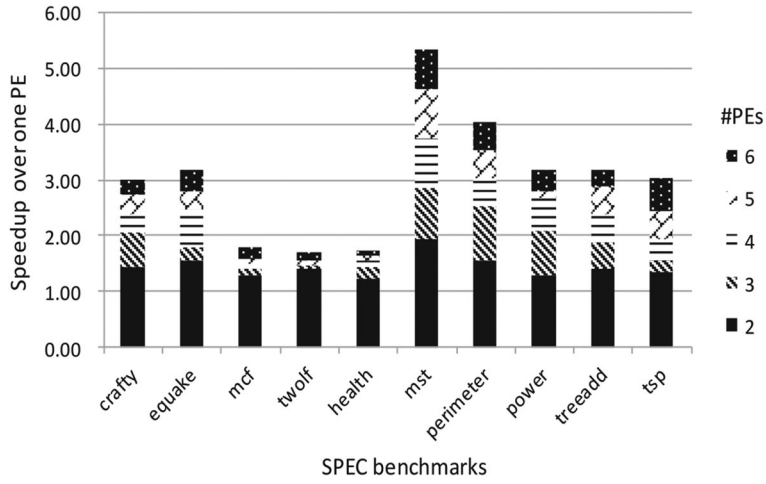

Fig. 13 Speedups of SPEC2020 benchmarks with HR-based approach in prophet

6.2.2 Comparison with best-found performance

Machine learning can learn prior knowledge from known facts and perform well on unseen instances. Since a program has a large number of partition structures, it is very time consuming to find the known facts in SpMT. In our work, we generate 414,687 different partition structures for every program and select the best partition structure as a training sample that is called "Best-Found" performance.

Compared with HR-based approach, IDaTPA performs well and we wish to know whether or not there is further improvement room by assessing how close the Best-Found is to IDaTPA's performance.

In Fig. 11, we compare IDaTPA with the Best-Found. For *power*, *mst* and *treeadd*, they almost reach this Best-Found performance. But for other programs such as *bh*, *em3d*, *perimeter*, *bisort*, *voronoi*, *tsp*, and *health*, there is significant improvement room. Although IDaTPA outperforms prior HR-based approach on these programs, it could have done better. Overall we achieve 90.64% of that maximum performance and the results show the programs' satisfied partitioning strategies vary in accordance with their characteristics.

These examples indicate that programs with different characteristics apply different partitioning strategies. Essentially, the HR-based approach is a strategy of one-size-fits-all. By using a machine learning technique, IDaTPA handles this problem. It uses prior knowledge to choose and apply the program-specific partitioning strategy according to target programs' characteristics, yielding a better performance than hardwired heuristics.

6.2.3 Speedups of SPEC2020 benchmarks with IDaTPA

Figure 12 indicates the general speedups with IDaTPA for part of SPEC2020 benchmarks. X-axis denotes selected benchmarks, including *164.gzip*, *176.gcc*, *177.mesa*, *181.mcf*, *188.ammp*, *183.equake*, *186.crafty*, *197.parser*, *253.perlbmk*, *254.gap*, *255.vertex*, *256.bzip2*, *300.twolf*, while speedups after using IDaTPA are shown along Y-axis. The benchmarks:*254.gap*, *186.crafty* and *197.parser* deliver significant performance improvement while *177.mesa*, *300.twolf*, *183.equake*, *188.ammp*, *255.vertex*, *256.bzip2* have little performance improvement.

Figure 13 shows the speedups of IDaTPA for SPEC2020 benchmarks in Prophet. In Fig. 13, X-axis indicates Olden and SPEC2020 benchmarks, while Y-axis shows the speedups over one PE (processing element). With the support of Prophet [26], IDaTPA is performed upon it, and gets speedups over one PE.

6.2.4 Portability of IDaTPA

In this section, we verify the portability of IDaTPA on the Prophet simulator with 8-core (a new platform). On the 8-core platform, training samples are re-collected and used to construct the machine learning model. Note that the same programs are applied to training IDaTPA which is used on the 8-core platform.

Figure 14 shows the speedups of HR-based approach and IDaTPA on the 8-core platform. Compared with the HR-based approach, IDaTPA shows a more stable and better performance improvement. Figure 14 indicates that all

Table 8 Olden benchmarks' dynamic information statistics

Olden benchmarks	Spawned threads		Success ratio		Successful threads	
	HR-based	IDaTPA	HR-based	IDaTPA	HR-based	IDaTPA
bh	122,761	121,862	0.95	0.97	116,623	118,206
em3d	3787	3791	0.87	0.87	3295	3298
health	3684	4252	0.95	0.91	3500	3869
perimeter	14971	21,296	0.49	0.52	7336	11074
voronoi	109,714	110,793	0.94	0.94	103,131	104,145
treeadd	2562	3079	0.40	0.58	1025	1786
power	121,419	122,051	0.85	0.85	103206	103,743
tsp	197,824	182,394	0.68	0.79	134520	144,091
mst	498	518	0.88	0.95	438	492
bisort	60,507	42,721	0.57	0.83	34489	35458
mean	63,773	61,276	0.76	0.82	48467	50246

Olden benchmarks	Squash rate		Performance improvement
	HR-based (%)	IDaTPA (%)	
bh	4.64	3.10	1.78
em3d	12.94	12.98	2.66
health	4.59	8.70	15.34
perimeter	51.26	48.26	15.89
voronoi	6.15	6.49	1.01
treeadd	60.02	41.67	6.88
power	15.11	14.65	1.19
tsp	31.71	21.39	2.89
mst	11.85	5.02	3.42
bisort	42.56	17.38	6.31
mean	20.26	3.10	5.74

Table 9 Olden benchmarks' thread statistics

Olden benchmarks		*em3d*	*Health*	*Perimeter*	*Voronoi*	*Treeadd*	*Power*	*tsp*
HR-based	Thread size	38.33	49.7	39.4	101.82	34	62.47	30.25
	Live-ins	3	3.4	1	2.14	1	3.448	0.80
	Slice/thread	0.10	0.09	0.05	0.03	0.06	0.083	0.0008
IDaTPA	Thread size	36.31	35.06	20.44	55.73	19.33	32.59	201.1
	Live-ins	2.54	2.5	2	1.8	1.33	3.261	0.38
	Slice/thread	0.10	0.10	0.15	0.05	0.12	0.11	0.0009

Olden benchmarks		*mst*	*bisort*	*bh*	*Mean*	*Variance*
HR-based method	Thread size	40	27.83	38.84	46.26	479.11
	Live-ins	3.32	2.5	3	2	3.12
	Slice/thread	0.07	0.12	0.10	0.11	0.11
IDaTPA	Thread size	57.81	18.85	23.53	28	30.83
	Live-ins	3.63	1.76	2.37	2.33	2.35
	Slice/thread	0.08	0.15	0.14	0.12	0.10

Table 10 The dynamic instruction statistics of olden benchmarks

Olden	*em3d (%)*	*Health (%)*	*Perimeter (%)*	*Voronoi (%)*	*Treeadd (%)*	*Power (%)*	*tsp (%)*	*mst (%)*	*bisort (%)*	*bh (%)*
Non-loop	28	22.2	100	55.8	100	40.5	59.2	55.5	73.7	47.3
Loop	72	77.8	0	44.2	0	59.5	40.8	44.5	26.3	52.7

programs yield higher speedup improvement with different percentages and the increasing rates of performance improvement vary from 1.12% to 15.89%, especially program *health*, *perimeter*, *bisort*, *mst*, *treeadd* have improved performance significantly. Two important factors lead to this significant performance gain: firstly, IDaTPA provides the satisfied thread partitioning scheme for every program according to their features; secondly, because these irregular programs have more inherent parallelism, the parallelism can be adequately exploited in the case of enough core resources. Program *power* exhibits a meager increase. Although IDaTPA can predict the best partitioned structures for these programs, which have complex control and data dependence. So even if processor resources are sufficient, these speculative threads are withdrawn frequently and cannot make effective use of core resources. Consequently, the speedups of these programs with complex dependence do not change when core number increases to a certain number.

Although IDaTPA delivers a better performance improvement on 8-core, however, just like in the case 4-core, there are some room to improvement compared with Best-Found.

7 Conclusion and perspectives

7.1 Conclusion

On the Prophet, this paper proposes an importance degree based thread partitioning approach (IDaTPA), to realize the partition of irregular programs. According to the characteristics of the program, the thread partition scheme is predicted, and the programs are partitioned according to the predicted partition scheme. Finally, Prophet simulator is used to verify its execution performance. The primary researches are listed as follows:

7.1.1 Construction of sample set

This paper proposes to construct a TLS sample set, which is composed of feature and partition scheme. The features are extracted along the CFG's speculative path in every program's procedure. For the method of partitioning scheme's acquisition in TLS samples, an expert experience based explicit partition is used to explicitly partition the program, and every procedure's satisfied partition scheme in the program is calculated.

7.1.2 Thread partition based on programs' characteristics

This paper applies a KNN classification algorithm in machine learning to generating a better partition scheme for every procedure to be partitioned from the sample set.

7.1.3 Prediction of partition scheme

This paper proposes an IDaTPA to partition programs with a KNN method. This paper performs nonloop partition and loop partition respectively. An evaluation model is established for the programs' nonloops, and nonloops are iteratively partitioned in accordance with the evaluation model. For the loops, the inner loops and the nested iterations are

Table 11 Olden benchmarks' characteristics

Olden	*bh*	*em3d*	*health*	*perimeter*
Main pointer	Heterogenous	Single-linked	Double-linked	Quadtree
Data Structures	Octree	lists	lists Quadtree	Quadtree
Input Parameters	64-2	10-3-30	2-16-1	4
#excuted /sum	24/28	13/15	13/14	0/0
Average number of iterations	9.14	8.73	3.51	0
Loop body size	200.45	265.99	172.88	0
Olden	*treeadd*	*power*	*tsp*	*mst*
Main pointer	Binary tree	Multiway tree	Binary tree	Array of singly
Data structure	Binary tree	Single-linked	Linked lists	linked lists
Input parameter	8-4-1	1-1	100-1	8-1
#excuted/sum	0/0	17/20	2/7	11/13
Average number of iterations	0	2.88	94.5	3.92
Loop body size	0	2625.27	1267.25	830.59
Olden	*voronoi*			
Main pointer	Binary			
Data structures	tree			
Input parameters	128-1-0			
#excuted /sum	3/4			
Average number of iterations	1.3			
Loop body size	137			
Olden	*bisort*			
Main pointer	Binary tree			
Data structure	Binary tree			
Input parameter	512-1			
#excuted/sum	2/3			
Average number of iterations	5.87			
Loop body size	41.42			

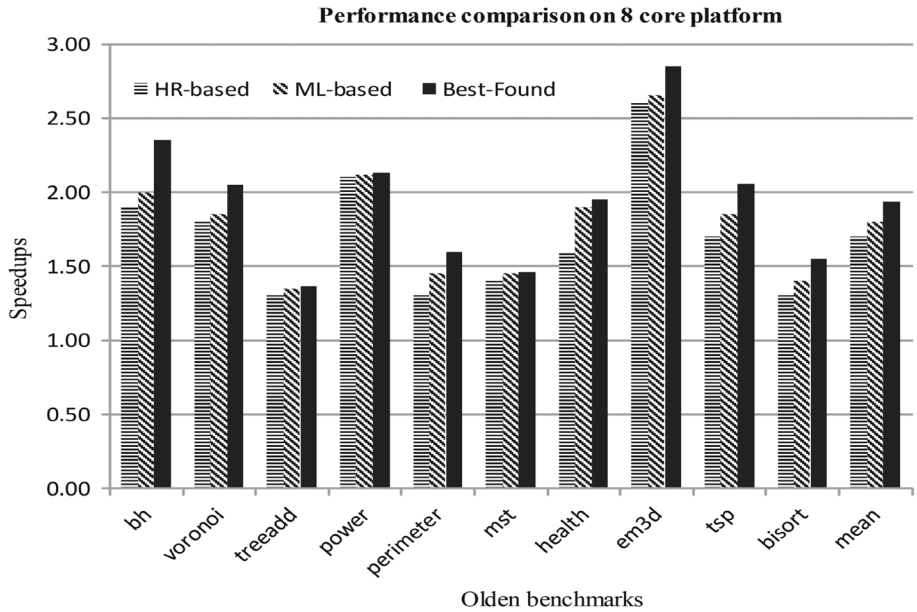

Fig. 14 Performance comparison on 8 core platform

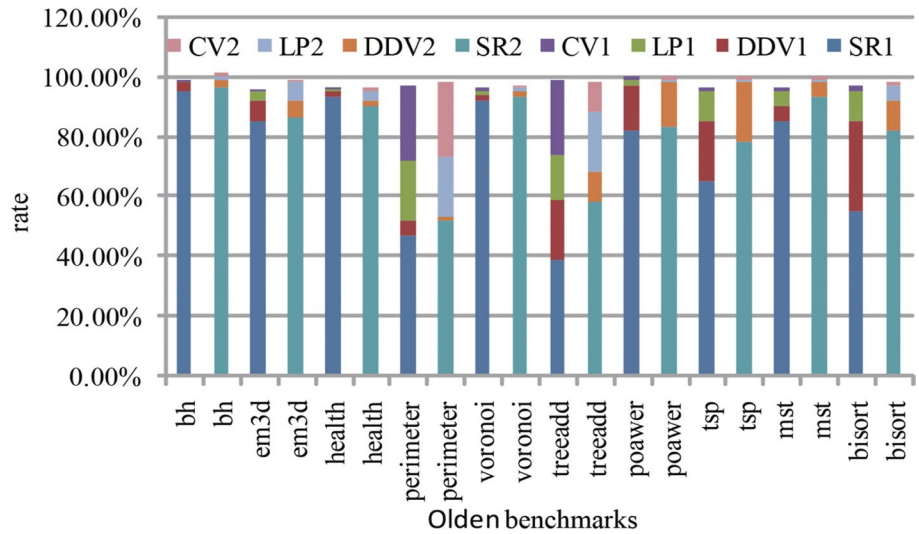

Fig. 15 Breakdown of speculative execution

respectively partitioned; for nested loops, nested iteration and sequential spawning are used for thread partitioning; for the inner loops, nonloop partitioning method is used for thread partitioning.

7.1.4 Validation

Using Olden and SPEC2020 benchmarks, the effect of IDaTPA is verified on Prophet, and is compared to the original partition results. The test results indicate that the IDaTPA averagely delivers a speedup of 1.80 on a 4-core processor.

7.2 Future work

TLS has been evolving many years, showing great advantages in making use of multicore resources. IDaTPA is proposed to handle the issue that conventional partitioning approaches can not generate the best partitioning scheme

for unknown programs. The trend of thread partition falls on two points: 1. partition process will be facilitated on foundation of procedures of programs; 2. IDaTPA will be implemented on hardwares.

7.3 Perspectives

As to the computing platform, the implementation of TLS is gradually turning from single chip multi-core component to distributed platform, like thread level speculative parallelism of decompression algorithm in [29] on Spark platform. To overcome the limitation that simulated annealing algorithm (SA) has low efficiency as the conventional SA algorithms still run on new platforms with low parallelism and the computing resource can not be utilized adequately, Wang et al. [30] expands the parallelism of algorithms to improve algorithms' efficiency by raising a speculative parallel SA algorithm based on Apache Spark.

Acknowledgements We thank our 505 and 509 laboratory for their great support during our work. We give our sincere wishes to all our colleagues of laboratory for their collaboration. We also thank reviewers for their careful comments.

Author contributions Yuxiang Li designed the approach, performed the software, wrote the methodology, experimental results and the literature review. Yuxiang Li reviewed the methodology and experimental results and provided valuable ideas for better article framework. Yuxiang Li and Xinyong Wang double checked the manuscript. All authors read and approved the final manuscript.

Funding National Natural Science Foundation of China Grant No.61972133 and No.12101195, Project of Leading Talents in Science and Technology Innovation for Thousands of People Plan in Henan Province Grant No. 204200510021, Henan Province Natural Science Fund Grant No. 202300410146 and No.232300420148, Henan Province Key Scientific and Technological Projects Grant No.222102210177, No.212102210383, No.202102210162 and No.222102210072 and No.242102210140, and China Postdoctoral Science Foundation Grant No.2021M700885, and the Key Research and Development Plan Special Project of Henan Province Grant No.241111211400.

Data availability The data that support the findings of this study are available from the corresponding author, upon reasonable request.

Declarations

Competing interests The authors declare that they have no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Li Y, Zhao Y, Liu B. Qinling: a parametric model in speculative multithreading. Symmetry. 2017;9(9):180.
2. Estebanez A, Llanos DR, Gonzalez-Escribano A. A survey on thread-level speculation techniques. ACM Comput Surv (CSUR). 2016;49(2):22.
3. Wang Y, Watling R, Qiu J, Wang Z. Gspecpal: speculation-centric finite state machine parallelization on gpus. In: 2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS); 2022. p. 481–91. https://doi.org/10.1109/IPDPS53621.2022.00053.
4. Wang Y, Watling R, Qiu J, Wang Z. Gspecpal: speculation-centric finite state machine parallelization on gpus. In: 2022 IEEE 36th international parallel and distributed processing symposium (IPDPS 2022). International Parallel and Distributed Processing Symposium IPDPS; 2022. p. 481–91. https://doi.org/10.1109/IPDPS53621.2022.00053. IEEE; IEEE Comp Soc. 36th IEEE International Parallel and Distributed Processing Symposium (IEEE IPDPS), ELECTR NETWORK, May 30-Jun 03, 2022.
5. Franklin M. Multiscalar processors. Acm Sigarch Comput Arch News. 1995;23(2):414–25.
6. Luo Y, Zhai A. Dynamically dispatching speculative threads to improve sequential execution. Acm Trans Arch Code Optim. 2012;9(3):732–41.
7. Sun X-H, Chen Y. Reevaluating Amdahl's law in the multicore era. J Parallel Distrib Comput. 2010;70(2):183–8.
8. Vijaykumar T, Sohi GS. Task selection for a multiscalar processor. In: Proceedings of the 31st Annual ACM/IEEE International Symposium on Microarchitecture, IEEE Computer Society Press; 1998. p. 81–92.
9. Gao L, Li L, Xue J, Yew P-C. Seed: a statically greedy and dynamically adaptive approach for speculative loop execution. IEEE Trans Comput. 2013;62(5):1004–16.
10. Sharafeddine M, Jothi K, Akkary H. Disjoint out-of-order execution processor. ACM Trans Arch Code Optim. 2012;9(3):19.

11. August DI, Huang J, Beard SR, Johnson NP, Jablin TB. Automatically exploiting cross-invocation parallelism using runtime information. In: Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). IEEE Computer Society; 2013. p. 1–11.
12. Hammond L, Hubbert BA, Siu M, Prabhu MK, Chen M, Olukolun K. The stanford hydra cmp. IEEE Micro. 2000;20(2):71–84.
13. Ohsawa T, Takagi M, Kawahara S, Matsushita S. Pinot: speculative multi-threading processor architecture exploiting parallelism over a wide range of granularities. In: Proceedings of the 38th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE Computer Society; 2005. p. 81–92.
14. Liu W, Tuck J, Ceze L, Ahn W, Strauss K, Renau J, Torrellas J. Posh: a tls compiler that exploits program structure. In: Proceedings of the Eleventh ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. ACM; 2006. p. 158–67.
15. Madriles C, García-Quiñones C, Sánchez J, Marcuello P, González A, Tullsen DM, Wang H, Shen JP. Mitosis: a speculative multithreaded processor based on precomputation slices. IEEE Trans Parallel Distrib Syst. 2008;19(7):914–25.
16. Wang S, Yew P-C, Zhai A. Code transformations for enhancing the performance of speculatively parallel threads. J Cir Syst Comput. 2012;21(02):1240008.
17. Zhai A, Colohan CB, Steffan JG, Mowry TC. Compiler optimization of memory-resident value communication between speculative threads. In: Proceedings of the International Symposium on Code Generation and Optimization: feedback-directed and Runtime Optimization. IEEE Computer Society; 2004. p. 39.
18. Zheng B, Tsai J-Y, Zang B, Chen T, Huang B, Li J, Ding Y, Liang J, Zhen Y, Yew P-C, et al. Designing the agassiz compiler for concurrent multithreaded architectures. In: International Workshop on Languages and Compilers for Parallel Computing. Springer; 1999. p. 380–398.
19. Johnson TA, Eigenmann R, Vijaykumar T. Min-cut program decomposition for thread-level speculation. In: ACM Sigplan Notices, vol. 39. ACM; 2004. p. 59–70.
20. Sohi G. Multiscalar: another fourth-generation processor. Computer. 1997;30(9):72–72.
21. Bhowmik A, Franklin M. A general compiler framework for speculative multithreading. In: Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures. ACM; 2002. p. 99–108.
22. Chen M, Olukotun K. Test: a tracer for extracting speculative threads. In: Code Generation and Optimization, 2003. CGO 2003. International Symposium On. IEEE; 2003. p. 301–312.
23. Hammond L, Hubbert BA, Siu M, Prabhu MK. The stanford hydra cmp. IEEE Micro. 2000;20(2):71–84.
24. Steffan JG, Colohan C, Zhai A, Mowry TC. The stampede approach to thread-level speculation. ACM Trans Comput Syst. 2005;23(3):253–300.
25. Quiñones CG, Madriles C, Sánchez J, Marcuello P, González A, Tullsen DM. Mitosis compiler: an infrastructure for speculative threading based on pre-computation slices. In: ACM Sigplan Notices, vol. 40. ACM; 2005. p. 269–279.
26. Chen Z, Zhao Y-L, Pan X-Y, Dong Z-Y, Gao B, Zhong Z-W. An overview of prophet. In: International Conference on Algorithms and Architectures for Parallel Processing. Springer; 2009. p. 396–407.
27. Song S, Zhao Y, Feng B, Wei Y, Wang X, Zhao H. Prophet+: an extended multicore simulator for speculative multithreading. J Xian Jiaotong Univ. 2010;44(10):13–5.
28. Dong Z, Zhao Y, Wei Y, Wang X, Song S. Prophet: a speculative multi-threading execution model with architectural support based on cmp. In: Scalable Computing and Communications; Eighth International Conference on Embedded Computing, 2009. SCALCOM-EMBEDDEDCOM'09. International Conference On. IEEE; 2009. p. 103–108.
29. Wang Z, Zhao Y, Liu Y, Chen Z, Lv C, Li Y. A speculative parallel decompression algorithm on apache spark. J Supercomput. 2017;73(9):1–30.
30. Wang Z, Zhao Y, Liu Y, Lv C. A speculative parallel simulated annealing algorithm based on apache spark. Concurr Comput Pract Exp. 2018;1:4429.
31. Li Y, Zhao Y, Sun L, Shen M. Optimizing partition thresholds in speculative multithreading. Icic Express Lett. 2017;11(6):1053–61.
32. Li Y, Zhao Y, Shi J. A hybrid samples generation approach in speculative multithreading. In: High Performance Computing and Communications; IEEE 14th International Conference on Smart City; IEEE 2nd International Conference on Data Science and Systems (HPCC/SmartCity/DSS), 2016 IEEE 18th International Conference On. IEEE; 2016. p. 35–41.
33. Liu B, Zhao Y, Li Y, Sun Y, Feng B. A thread partitioning approach for speculative multithreading. J Supercomput. 2014;67(3):778–805.
34. Liu B, Zhao Y, Li M, Liu Y, Feng B. A virtual sample generation approach for speculative multithreading using feature sets and abstract syntax trees. In: 2012 13th International Conference on Parallel and Distributed Computing, Applications and Technologies. IEEE; 2012. p. 39–44.
35. Li D-C, Lin Y-S. Using virtual sample generation to build up management knowledge in the early manufacturing stages. Eur J Oper Res. 2006;175(1):413–34.
36. Li Y, Zhao Y, Wu Q. Gba:a graph-based thread partition approach in speculative multithreading. Concurr Comput Pract Exp. 2017;29(21):4294.
37. Grewe D, Wang Z, O'Boyle MF. A workload-aware mapping approach for data-parallel programs. In: Proceedings of the 6th International Conference on High Performance and Embedded Architectures and Compilers. ACM; 2011. p. 117–26.
38. Long S, Fursin G, Franke B. A cost-aware parallel workload allocation approach based on machine learning techniques. In: IFIP International Conference on Network and Parallel Computing. Springer; 2007. p. 506–515.
39. Tournavitis G, Wang Z, Franke B, O'Boyle MF. Towards a holistic approach to auto-parallelization: integrating profile-driven parallelism detection and machine-learning based mapping. ACM Sigplan Notices. 2009;44(6):177–87.
40. Wang Z, O'Boyle MFP. Mapping parallelism to multi-cores: a machine learning based approach. In: ACM Sigplan Symposium on Principles and Practice of Parallel Programming, PPOPP 2009, Raleigh, Nc, Usa, February; 2009. p. 75–84.
41. Chen X, Long S. Adaptive multi-versioning for openmp parallelization via machine learning. In: Parallel and Distributed Systems (ICPADS), 2009 15th International Conference On. IEEE; 2009. p. 907–912.
42. Wang Z, O'Boyle MF. Partitioning streaming parallelism for multi-cores: a machine learning based approach. In: Proceedings of the 19th International Conference on Parallel Architectures and Compilation Techniques. ACM; 2010. p. 307–318.
43. Singer J, Yiapanis P, Pocock A, Lujan M, Brown G, Ioannou N, Cintra M. Static java program features for intelligent squash prediction. Statistical and Machine learning approaches to ARchitecture and compilaTion (SMART 10). 2010;14.
44. Xupeng M, Yujie W, Jia S, Yuxia S, Bin C. Acceleration of graph neural network training for multiple gpus. J Softw. 2023;34(9):4407–20.

45. Sarkar V, Hennessy J. Partitioning parallel programs for macro-dataflow. In: Proceedings of the 1986 ACM Conference on LISP and Functional Programming. ACM; 1986. p. 202–11.
46. Shinmura S. The 100-fold cross validation for small sample method. Data Anal. 2016;2016:41.
47. Guo G, Wang H, Bell D, Bi Y, Greer K. Knn model-based approach in classification. Lect Notes Comput Sci. 2003;2888:986–96.
48. Li Y, Zhao Y, Sun L, Shen M. A hybrid sample generation approach in speculative multithreading. J Supercomput. 2017;3:1–33.
49. Monsifrot A, Bodin F, Quiniou R. A machine learning approach to automatic production of compiler heuristics. In: International Conference on Artificial Intelligence: Methodology, Systems, and Applications. Springer; 2002. p. 41–50.
50. Olden B. benchmark suite v; 2010.
51. Prabhu MK, Olukotun K. Exposing speculative thread parallelism in spec2000. In: Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. ACM; 2005. p. 142–152.
52. Anderson DR. Taylor's formula and integral inequalities for conformable fractional derivatives. In: Contributions in mathematics and engineering. Berlin: Springer; 2016. p. 25–43.
53. De Cuypere E, De Turck K, Wittevrongel S, Fiems D. A maclaurin-series expansion approach to coupled queues with phase-type distributed service times. In: Proceedings of the 10th EAI International Conference on Performance Evaluation Methodologies and Tools. ACM; 2016.
54. Mullin MD, Sukthankar R. Complete cross-validation for nearest neighbor classifiers. In: ICML; 2000. p. 639–46.
55. Wilson RP, French RS, Wilson CS, Amarasinghe SP, Anderson JM, Tjiang SW, Liao S-W, Tseng C-W, Hall MW, Lam MS, et al. Suif: an infrastructure for research on parallelizing and optimizing compilers. ACM Sigplan Notices. 1994;29(12):31–7.
56. Rogers A, Carlisle MC, Reppy JH, Hendren LJ. Supporting dynamic data structures on distributed-memory machines. ACM Trans Program Lang Syst (TOPLAS). 1995;17(2):233–63.
57. Yuxiang L, Yinliang Z, Huan G. Thread partition prediction model based on artificial neural network; 2017.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.